JOB PORTAL (WEB APPLICATION)

MINI PROJECT REPORT

Submitted by

UTKARSH SRIVASTAVA
[EA*****010355]**

Under the Guidance of
Dr. G. Babu

(Assistant Professor, Directorate of Online Education)

in

partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

DIRECTORATE OF ONLINE EDUCATION
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

November 2025

DIRECTORATE OF ONLINE EDUCATION

SRM INSTITUTE OF SCIENCE AND

TECHNOLOGY KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

This Mini Project Work Report titled “Job Portal” Utkarsh Srivastava [EA2432251010355], who carried out the Mini Project Work under my supervision along with the company mentor. Certified further, that to the best of my knowledge the work reported herein does not form any other internship report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

CERTIFICATE OF MINI PROJECT WORK

This is to certify that **Utkarsh Srivastava** [**EA2432251010355]** has undergone mini project work during the period from **10-08-2025 to 12-11-2025** under the guidance of **Dr. G. Babu**, Assistant Professor, Directorate of Online Education.

This work has been carried out in partial fulfilment of the requirements for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**.

Abstract

The “Job Portal (Web Application)” is designed to provide a common platform for job seekers and employers. The main objective of this project is to simplify the recruitment process by enabling employers to post job openings and candidates to search for and apply for suitable opportunities online.

The system allows job seekers to register, create and manage their profiles, upload resumes, and search for jobs based on skills, location, and domain. Employers can

register, create company profiles, and post job vacancies with detailed requirements. The application also supports job applications tracking and basic communication between candidates and employers.

Technologies used for the development include HTML, CSS, JavaScript with React.js as a framework for the frontend, Node.js for backend, and MySQL as the database. This project demonstrates -

real-world software development practices such as authentication, CRUD operations, and database connectivity.

In the future, the system can be enhanced with advanced features like AI-based job recommendations, resume parsing, and email notifications.

Contents

Introduction

In today's competitive world, the recruitment process has become a crucial activity for both employers and job seekers. Traditional methods of job searching and hiring, such as newspaper advertisements or offline recruitment agencies, are often time-consuming, inefficient, and limited in reach. To overcome these challenges, online job portals have emerged as a practical solution, providing a digital platform that connects job seekers with potential employers.

The proposed "Job Portal (Web Application)" aims to simplify and streamline the hiring process by enabling employers to post job vacancies and candidates to search, apply, and track opportunities online. The system is designed with a user-friendly interface, ensuring easy accessibility for both parties. Job seekers can create profiles, upload resumes, and search for jobs based on criteria such as skills, domain, and location, while employers can manage company profiles and post job requirements effectively.

This project demonstrates the application of modern software development techniques, integrating frontend, backend, and database technologies. It not only provides a basic recruitment platform but also lays the foundation for advanced features such as AI-based job recommendations, resume parsing, and secure communication between candidates and employers.

By implementing this system, the recruitment process becomes faster, more transparent, and accessible, thus benefiting both job seekers and organizations in achieving their objectives.

Analysis and Requirement

1. System Analysis

The Job Portal (Web Application) is designed to bridge the gap between job seekers and employers by providing a common online platform for recruitment. The existing recruitment process often involves offline applications, limited accessibility, and time delays. By shifting to an online system, the process becomes faster, easier, and more efficient.
The system will allow job seekers to create and manage profiles, upload resumes, and apply for jobs online. Employers can register, manage company profiles, and post job opportunities. The administrator will have full control to monitor, verify, and manage the activities on the portal.

2. Requirement Analysis

2.1 Functional Requirements

- User Registration & Login: Separate modules for job seekers, employers, and administrator.
- Profile Management: Job seekers can manage resumes, while employers can update company information.
- Job Posting & Management: Employers can post, edit, and delete jobs.
- Job Search & Apply: Job seekers can search based on title, skill, and location, and apply for jobs.

2.2 Non-Functional Requirements

- Usability: The system should have a user-friendly interface.
- Performance: The portal should handle multiple concurrent users efficiently.
- Security: Secure authentication and data protection for users.
- Scalability: The system should support future expansion and integration of advanced features.
- Reliability: Ensure data accuracy and system availability.

3. Hardware and Software Requirements

• Hardware Requirements:
- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 500 GB

• Software Requirements:
- Operating System: Windows / Linux
- Frontend: HTML, CSS, JavaScript
- Backend: Node.js
- Database: MySQL
- Web Browser: Chrome / Firefox / Edge

Problem Description and Modules Description

1. **Problem Description**
   In the traditional recruitment process, job seekers rely on newspapers, employment agencies, or personal networks to find opportunities, while employers often face difficulties in reaching suitable candidates. These methods are time-consuming, costly, and lack transparency. Additionally, job seekers cannot easily track their applications, and employers struggle with managing large volumes of resumes offline.
   To overcome these challenges, the **Job Portal (Web Application)** is proposed as an online solution. It provides a centralized platform where job seekers and employers can interact. Job seekers can register, create profiles, upload resumes, and apply for jobs. Employers can post vacancies, review applications, and manage company information. The system reduces manual effort, increases transparency, and enhances accessibility.

2. **Modules Description**

2.1 Job Seeker Module

- Register and login to the system.
- Create and manage profile including personal details, skills, and resume.
- Search for jobs by title, location, or skill.
- Apply for jobs and track application status.

2.2 Employer Module

- Register and login as an employer.
- Create and manage company profile.
- Post new job vacancies with detailed requirements.
- View applications and shortlist candidates.

2.3 Communication Module (Optional)

- Enable communication between job seekers and employers.
- Provide notifications or updates on job applications.
- Support for email alerts and system messages.

Implementation

The implementation phase converts the design into a functional system. The **Job Portal Web Application** was developed using modern web technologies ensuring scalability, maintainability, and usability.

1 Frontend Implementation

- The **user interface (UI)** is designed using **HTML**, **CSS**, and **JavaScript**.
- **Responsive design** ensures the portal works smoothly on desktops and mobile devices.
- The **frontend** communicates with the backend via HTTP requests using **AJAX / Fetch API**.

Key Features Implemented:

- User Registration and Login Pages
- Employer Dashboard for Job Posting
- Job Seeker Dashboard for Application Tracking
- Search and Filter Functionality
- Admin Panel for Management

---

2 Backend Implementation

- The backend handles business logic, data validation, and database operations.
- Developed using **PHP / Node.js / Django**.
- RESTful APIs were implemented for communication between the client and the server.
- Proper **authentication and authorization** were implemented using sessions/tokens.

Core Backend Modules:

1. **User Module:** Handles registration, login, and profile management.
2. **Job Module:** Manages job posting, updates, and deletions.
3. **Application Module:** Handles job applications and status tracking.
4. **Admin Module:** Allows monitoring and verification of activities.

Database Implementation

- **MySQL** is used as the relational database management system.
- The schema includes tables like:
    - users (user_id, name, email, password, role)
    - jobs (job_id, title, description, location, employer_id)
    - applications (app_id, user_id, job_id, status)
- Relationships are established through foreign keys to maintain data integrity.

Database Operations:

- CRUD (Create, Read, Update, Delete)
- Data validation at both application and database levels.

1. Design

System Architecture

The **Job Portal Web Application** follows a **three-tier architecture**, which divides the system into three logical layers:

1. **Presentation Layer (Frontend):**
    - Developed using **HTML, CSS, and JavaScript**.
    - Handles user interactions, registration, login, job search, and job posting interfaces.
    - Communicates with the backend via HTTP requests (GET/POST).
2. **Application Layer (Backend):**
    - Implemented using **PHP / Node.js / Django**.
    - Manages all the business logic such as authentication, job posting, and application tracking.
    - Performs validation and interacts with the database.
3. **Database Layer:**
    - Built on **MySQL**.
    - Stores information about users, jobs, companies, and applications.
    - Ensures data integrity and supports CRUD operations.

Architecture Diagram (Explanation)

Users (Job Seekers / Employers) → Frontend (HTML, CSS, JS) → Backend (PHP / Node.js / Django) → Database (MySQL)

↳ The Admin accesses all data via backend for monitoring and management.

---

2. Data Flow Diagram (DFD)

Level 0 DFD (Context Diagram)

Depicts the overall system interaction.

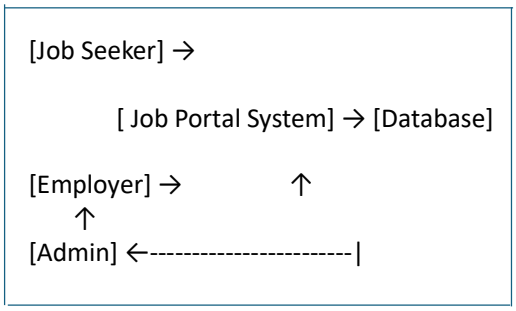
Entities:

Job Seeker – Registers, searches jobs, and applies.

Employer – Posts job openings, views applicants.

Data Flows:

- Job seekers send registration and job application data to the system.
- Employers send job postings and receive applications.
- The admin monitors activities and updates the database.



Level 1 DFD

Breaks down the internal processes of the system.

Processes:

- **User Registration & Authentication**
- **Profile Management**
- **Job Posting & Management**
- **Job Search & Application**
- **Admin Monitoring & Control**

Each process interacts with the database to fetch or update data.

---

3 UML Diagrams

(a) Use Case Diagram

Actors:

- Job Seeker
- Employer
- Admin

Use Cases:

- **Job Seeker:** Register, Login, Create Profile, Search Jobs, Apply Job, View Status.
- **Employer:** Register, Login, Post Job, Manage Job Listings, View Applicants.

[Job Seeker] —► (Register)

—► (Login)

—► (Search Jobs)

—► (Apply Job)

[Employer] —► (Post Job)

—► (View Applicants)

(b) Activity Diagram (Job Application Flow)

Start → User Login → Search Job → Select Job → Apply Job →

System Stores Application → Employer Reviews →

(Decision: Shortlisted)

→ Yes → Notify Candidate → End

→ No → Notify Rejection → End

(c) Class Diagram

Classes and Relationships:

Class Name	Attributes	Methods
User	user_id, name, email, password, role	register(), login(), updateProfile()
Job	job_id, title, description, location, salary, employer_id	postJob(), updateJob(), deleteJob()
Application	app_id, job_id, user_id, status	applyJob(), updateStatus()
Admin	admin_id, name, email	approveJob(), manageUser()

Relationships:

- User (1..*)* → *Application (0..*)
- Employer (1..*)* → *Job (0..*)
- Admin monitors all entities.

4 User Interface Design (Optional Section)

- **Login Page:** For Job Seeker, Employer, and Admin.
- **Dashboard:** Displays recent job postings and applications.
- **Job Posting Form:** For employers to add jobs.
- **Job Search Page:** For candidates with filters (skills, location, etc.)

1. Testing

Testing ensures that each component of the system works as expected and integrates correctly.

7.1 Testing Objectives

- To verify that the system performs all required functionalities accurately.
- To ensure the web application is reliable, secure, and user-friendly.

7.2 Types of Testing Conducted

1. **Unit Testing:**
   Each module (login, job posting, job search) was tested independently to verify correctness.

2. **Integration Testing:**
   Modules were integrated and tested together to ensure seamless interaction between frontend, backend, and database.

3. **System Testing:**
   The complete system was tested as a whole to validate overall functionality and performance.

4. **User Acceptance Testing (UAT):**
   Sample users (students and employers) tested the system to ensure it meets real-world requirements.

7.3 Test Cases Example

Test Case ID	Description	Input	Expected Output	Result
TC001	Verify User Registration	Valid details	Registration Successful	Pass
TC002	Verify Login Functionality	Correct credentials	Dashboard loads	Pass
TC003	Invalid Login	Wrong password	Error message displayed	Pass
TC004	Job Posting	Valid job data	Job added to list	Pass
TC005	Job Search	Keyword “Developer”	List of relevant jobs	Pass

---

8. Tools and Technologies Used

Category	Tools / Technologies
Frontend	HTML, CSS, JavaScript
Backend	Node.js
Database	MySQL
Server	Nginx
IDE / Editor	Visual Studio Code, Sublime Text
Version Control	Git, GitHub
Testing Tools	Postman (for API Testing), Browser Developer Tools
Operating System	Windows / Linux
Future Enhancements	AI-based recommendations, Resume Parsing, Email Notifications

Tools and Technologies Used

1. Frontend

- **HTML:** For structuring web pages.
- **CSS:** For styling and responsive design.
- **JavaScript:** For dynamic behavior and interactivity.

2. Backend

- **Node.js :** Handles server-side logic, authentication, job posting, and application tracking.
- **RESTful APIs:** Enables communication between frontend and backend.

3. Database

- **MySQL:** Stores user profiles, job postings, applications, and admin data.
- **Database Operations:** CRUD (Create, Read, Update, Delete) and relationships via foreign keys.

4. Server / Hosting

- **Nginx:** Hosts the web application and handles HTTP requests.

5. IDE / Editors

- **Visual Studio Code / Sublime Text:** Used for writing frontend and backend code.

6. Version Control

- **Git / GitHub:** For code versioning and collaboration.

7. Testing Tools

- **Postman:** API testing.
- **Browser Developer Tools:** Debugging and front-end validation.

8. Operating System

- **Windows / Linux:** Development and deployment environment.

9. Future Enhancements

- AI-based job recommendations.
- Resume parsing and automated matching.
- Email notifications for job updates and application status.

Appendices

Appendix A: Project Overview

The Job Portal App is a full-stack web application built using the **MERN (MongoDB, Express.js, React.js, Node.js)** stack. It connects job seekers and employers through an interactive and efficient platform. The system manages job listings, applications, and secure authentication for both users and employers.

Appendix B: Technologies Used

- **Frontend:** React.js with Vite JS framework
- **Backend:** Node.js with Express.js Framework MongoDB.
- **Authentication:** JWT (JSON Web Tokens), Bcrypt for password hashing
- **Image Management:** Cloudinary

Appendix C: System Requirements

- Node.js v20.9.0
- MongoDB Atlas account / local MongoDB server
- Cloudinary account for image storage
- Modern web browser (Chrome, Edge, or Firefox)

Appendix D: Installation & Configuration

1. Clone the repository and navigate into the project directory.
2. Install required NPM packages for both frontend and backend using npm install.
3. Configure environment variables in a config.env file inside the backend/config folder.
4. Start the backend using node server.js and frontend using npm run dev.
5. Access the application at http://localhost:3000

Appendix E: Key Features

- Secure user authentication using JWT
- Job posting and application tracking
- Responsive and interactive user interface
- Employer dashboard for managing postings
- Secure data handling and cloud deployment

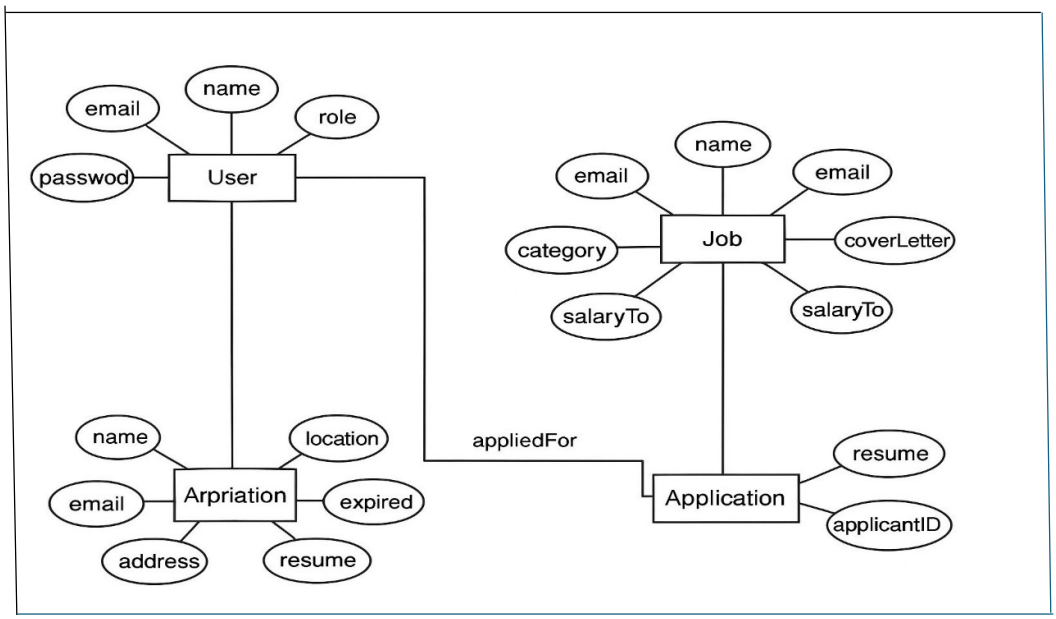
Appendix F: Contribution Guidelines

Developers can contribute by forking the repository, creating feature branches, committing changes, and submitting pull requests. Contributions help improve and expand the functionality of the platform.

Appendix G: Acknowledgment

This project demonstrates the practical application of full-stack web development principles using modern tools and frameworks.

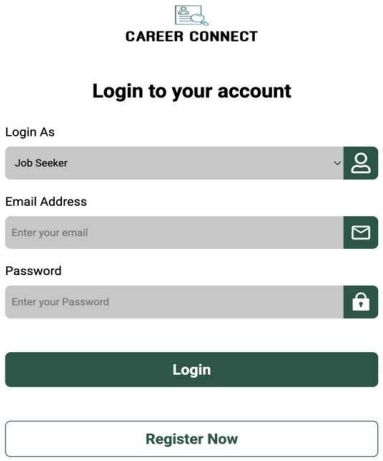
Er Diagram –



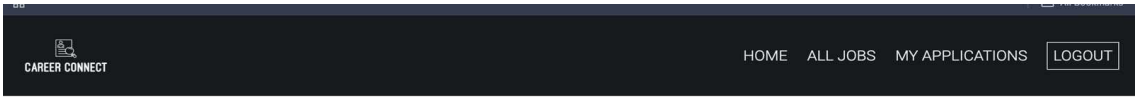
Project Screenshots –

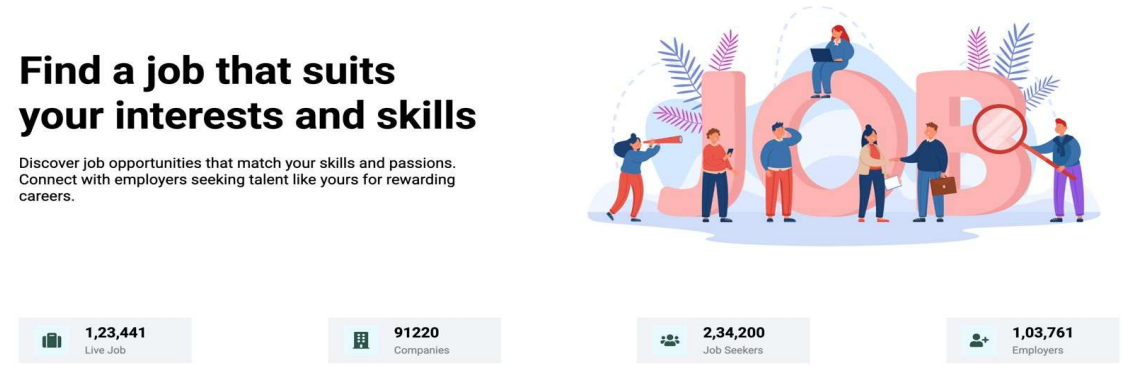
Jobseeker

- **Login**

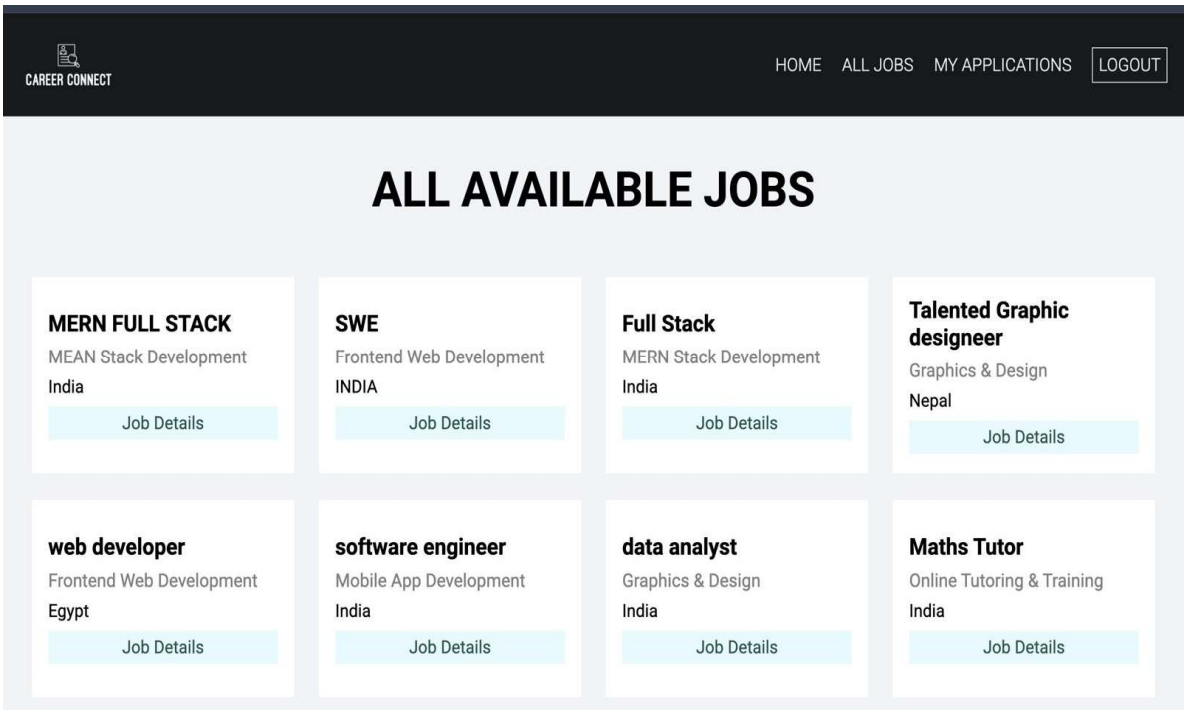


- **Home Page**

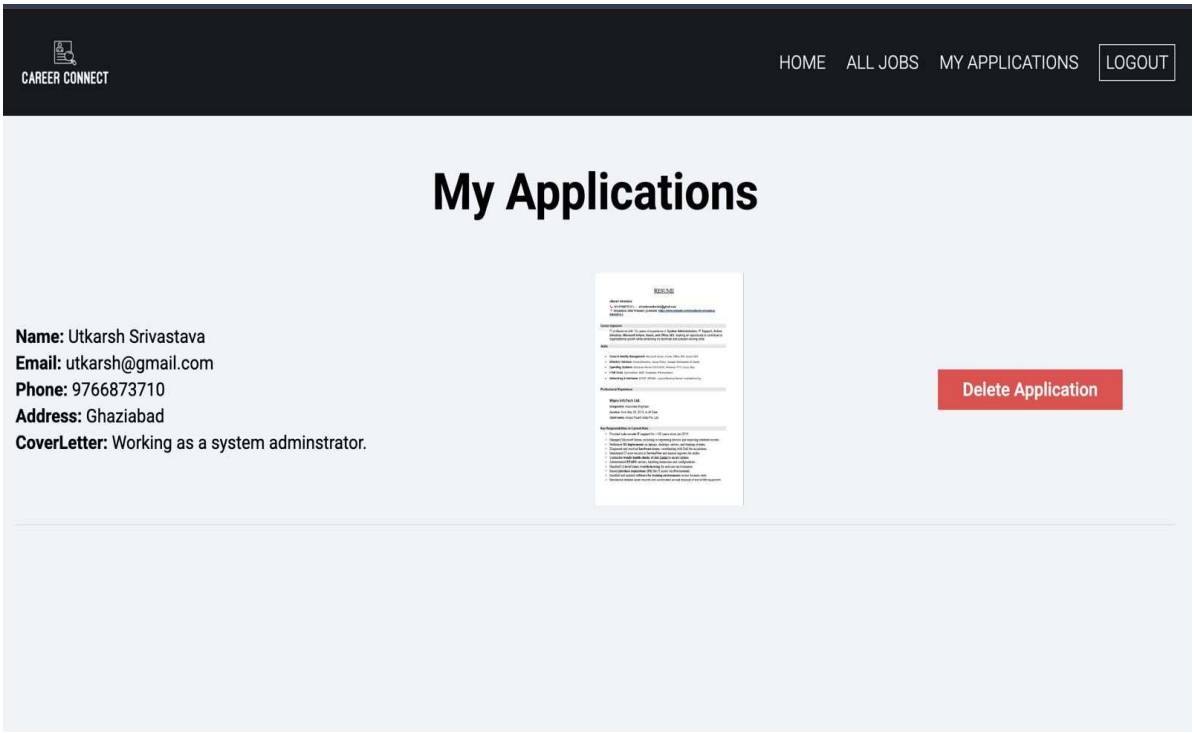




- **All Jobs**

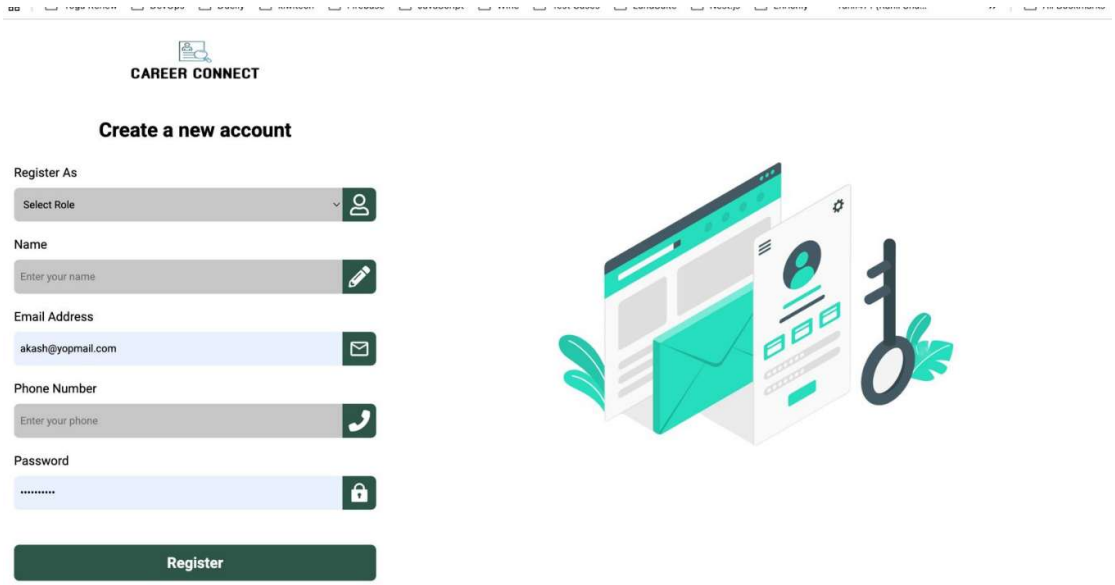


- **Job Applied**

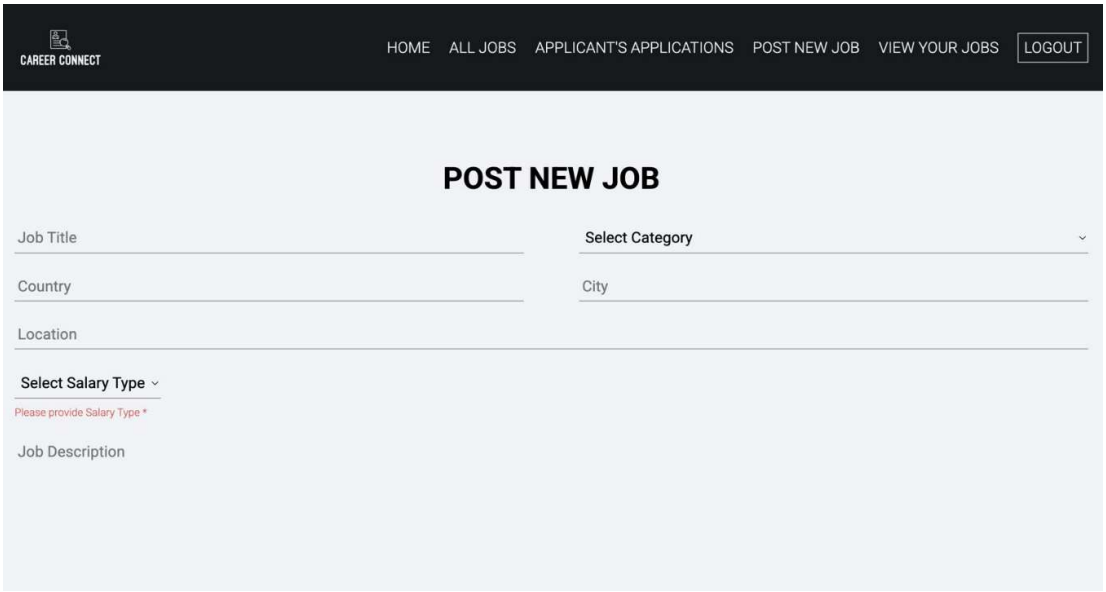


Recruiter

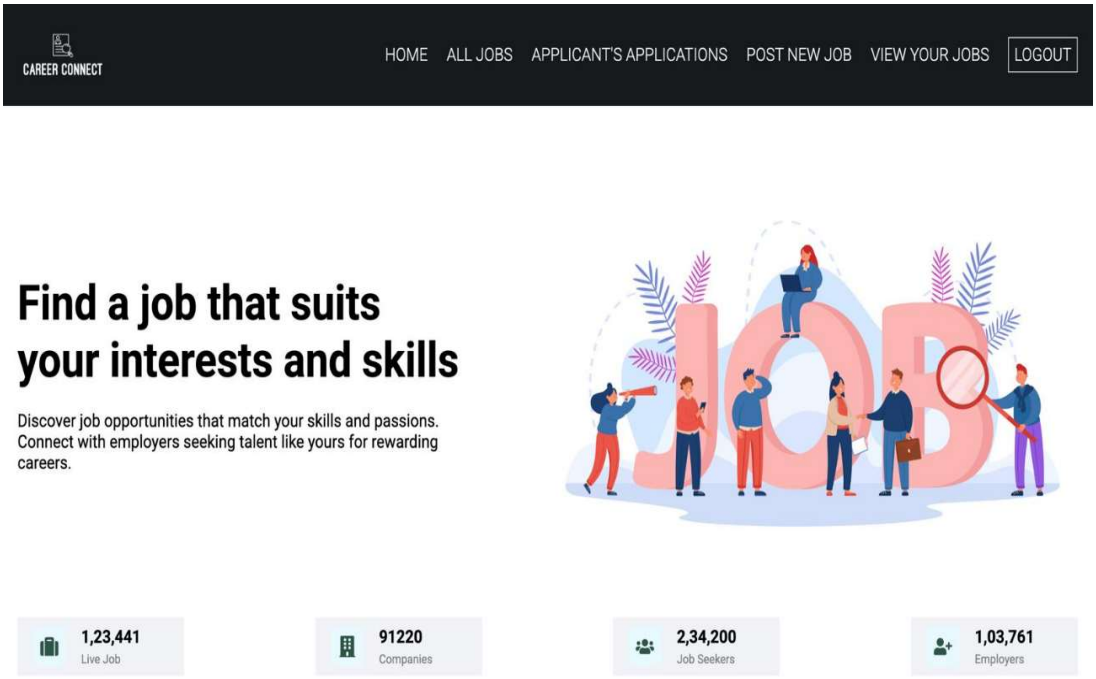
- **Sign-Up**



- **Recruiter Job Posting**



- **Home Page**


CAREER CONNECT
HOME
ALL JOBS
APPLICANT'S APPLICATIONS
POST NEW JOB
VIEW YOUR JOBS
LOGOUT
Find a job that suits your interests and skills
Discover job opportunities that match your skills and passions. Connect with employers seeking talent like yours for rewarding careers.
JOB
1,23,441
Live Job
91220
Companies
2,34,200
Job Seekers
1,03,761
Employers

Code Snippet –

Login Page

```jsx
import React, { useContext, useState } from "react";
import { MdOutlineMailOutline } from "react-icons/md";
import { RiLock2Fill } from "react-icons/ri";
import { Link, Navigate } from "react-router-dom";
import { FaRegUser } from "react-icons/fa";
import axios from "axios";
import toast from "react-hot-toast";
import { Context } from "../../main";

const Login = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [role, setRole] = useState("");

  const { isAuthorized, setIsAuthorized } = useContext(Context);

  const handleLogin = async (e) => {
    e.preventDefault();
    try {
      const { data } = await axios.post(
        "http://localhost:4000/api/v1/user/login",
        { email, password, role },
        {
          headers: {
            "Content-Type": "application/json",
          },
          withCredentials: true,
        }
      );
      toast.success(data.message);
      setEmail("");
      setPassword("");
      setRole("");
      setIsAuthorized(true);
    } catch (error) {
      toast.error(error.response.data.message);
    }
  };

  if(isAuthorized){
    return <Navigate to={'/'}/>
  }

  return (
    <>
      <section className="authPage">
        <div className="container">
          <div className="header">
            <img src="/careerconnect-black.png" alt="logo" />
            <h3>Login to your account</h3>
          </div>
          <form>
            <div className="inputTag">
              <label>Login As</label>
              <div>
```

```
            <div>
              <select value={role} onChange={(e) => setRole(e.target.value)}>
                <option value="">Select Role</option>

                <option value="Job Seeker">Job Seeker</option>
                <option value="Employer">Employer</option>
              </select>
              <FaRegUser />
            </div>
          </div>
          <div className="inputTag">
            <label>Email Address</label>
            <div>
              <input
                type="email"
                placeholder="Enter your email"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
              />
              <MdOutlineMailOutline />
            </div>
          </div>
          <div className="inputTag">
            <label>Password</label>
            <div>
              <input
                type="password"
                placeholder="Enter your Password"
                value={password}
                onChange={(e) => setPassword(e.target.value)}
              />
              <RiLock2Fill />
            </div>
          </div>
          <button type="submit" onClick={handleLogin}>
            Login
          </button>
          <Link to={"/register"}>Register Now</Link>
        </form>
      </div>
      <div className="banner">
        <img src="/login.png" alt="login" />
      </div>
    </section>
  </>
 );
};

export default Login;
```

Signup Page

```jsx
import React, { useContext, useState } from "react";
import { FaRegUser } from "react-icons/fa";
import { MdOutlineMailOutline } from "react-icons/md";
import { RiLock2Fill } from "react-icons/ri";
import { FaPencilAlt } from "react-icons/fa";
import { FaPhoneFlip } from "react-icons/fa6";
import { Link, Navigate } from "react-router-dom";
import axios from "axios";
import toast from "react-hot-toast";
import { Context } from "../../main";

const Register = () => {
  const [email, setEmail] = useState("");
  const [name, setName] = useState("");
  const [phone, setPhone] = useState("");
  const [password, setPassword] = useState("");
  const [role, setRole] = useState("");

  const { isAuthorized, setIsAuthorized, user, setUser } = useContext(Context);

  const handleRegister = async (e) => {
    e.preventDefault();
    try {
      const { data } = await axios.post(
        "http://localhost:4000/api/v1/user/register",
        { name, phone, email, role, password },
        {
          headers: {
            "Content-Type": "application/json",
          },
          withCredentials: true,
        }
      );
      toast.success(data.message);
      setName("");
      setEmail("");
      setPassword("");
      setPhone("");
      setRole("");
      setIsAuthorized(true);
    } catch (error) {
      toast.error(error.response.data.message);
    }
  };

  if(isAuthorized){
    return <Navigate to={'/'}/>
  }


  return (
    <>
      <section className="authPage">
        <div className="container">
          <div className="header">
            <img src="/careerconnect-black.png" alt="logo" />
            <h3>Create a new account</h3>
          </div>
          <form>
            <div className="inputTag">
              <label>Register As</label>
              <div>
                <select value={role} onChange={(e) => setRole(e.target.value)}>
                  <option value="">Select Role</option>
                  <option value="Employer">Employer</option>
                  <option value="Job Seeker">Job Seeker</option>
                </select>
```

```
            <FaRegUser />
          </div>
        </div>
        <div className="inputTag">
          <label>Name</label>
          <div>
            <input
              type="text"
              placeholder="Enter your name"
              value={name}
              onChange={(e) => setName(e.target.value)}
            />
            <FaPencilAlt />
          </div>
        </div>
        <div className="inputTag">
          <label>Email Address</label>
          <div>
            <input
              type="email"
              placeholder="Enter your email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
            />
            <MdOutlineMailOutline />
          </div>
        </div>
        <div className="inputTag">
          <label>Phone Number</label>
          <div>
            <input
              type="number"
              placeholder="Enter your phone"
              value={phone}
              onChange={(e) => setPhone(e.target.value)}
            />
            <FaPhoneFlip />
          </div>
        </div>
        <div className="inputTag">
          <label>Password</label>
          <div>
            <input
              type="password"
              placeholder="Enter your password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
            />
            <RiLock2Fill />
          </div>
        </div>
        <button type="submit" onClick={handleRegister}>
          Register
        </button>
        <Link to={"/login"}>Login Now</Link>
      </form>
    </div>
    <div className="banner">
      <img src="/register.png" alt="login" />
    </div>
  </section>
</>
  );
};

export default Register;
```

Conclusion

The Job Portal (Web Application) project provides an efficient and user-friendly platform that bridges the gap between job seekers and employers. It eliminates the limitations of traditional recruitment methods by digitizing the entire hiring process. Job seekers can easily search and apply for jobs, while employers can post vacancies and manage recruitment activities in a structured way.

The system leverages modern web technologies to ensure scalability, security, and accessibility. Overall, the Job Portal project successfully demonstrates how technology can simplify and enhance the recruitment process, making it faster, transparent, and more effective for both parties.

References

1. **MongoDB Documentation.** (2024). *NoSQL Database for Modern Applications.* Retrieved from https://www.mongodb.com/docs/

2. **Express.js Official Documentation.** (2024). *Fast, unopinionated, minimalist web framework for Node.js.* Retrieved from https://expressjs.com/

3. **React.js Documentation.** (2024). *A JavaScript library for building user interfaces.* Retrieved from https://react.dev/

4. **Node.js Documentation.** (2024). *Node.js JavaScript runtime built on Chrome's V8 JavaScript engine.* Retrieved from https://nodejs.org/en/docs

5. **Bootstrap Official Documentation.** (2024). *Front-end component library for responsive web design.* Retrieved from https://getbootstrap.com/docs/

6. **Cloudinary Developer Guide.** (2024). *Media management and optimization platform.* Retrieved from https://cloudinary.com/documentation

7. **JWT.io.** (2024). *Introduction to JSON Web Tokens for Secure Authentication.* Retrieved from https://jwt.io/introduction/

8. **MDN Web Docs.** (2024). *JavaScript and Web Development Tutorials and References.* Retrieved from https://developer.mozilla.org/

9. **GeeksforGeeks.** (2024). *MERN Stack Development Tutorial Series.* Retrieved from https://www.geeksforgeeks.org/mern-stack/

10. **FreeCodeCamp.** (2024). *Full Stack Web Development Guide using MERN Stack.* Retrieved from https://www.freecodecamp.org/

THANK YOU